```
//part A
//המנגנון מאוד דומה ל-TSL
//כאן פשוט צריך לשנות את הקבועים שעובדים איתם - במקום שיהיה
//TSL(lock, value)
//הפעולה מעלה מחזירה את הערך הקודם של המנעול ומשנה את ערכה לערך הדרוש
//ואז אפשר לשלב אותה בלולאה בצורה הבאה
while(TSL(lock, 1)) {
 //busy wait
 }
//כלומר כל עוד נעול אז ישאר נעול ויוחזר 1 ונשאר בלולאה
//וברגע שהוא פתוח אז הוא ינעל ויצא מהלולאה ואז אפשר להיכנס לקטע הקריטי
//אז במקום זאת יש לנו כאן טריק אחר
CAS(lock, expected, new_value)
//כאשר פשוט נציב את הערך המצופה כ-0 ואת הערך החדש כ-1 נקבל את אותה התנהגות
while(CAS(lock, 0, 1)){
 //busy wait
 }
//critical section
lock=1;

//part B

//כללית - הפתרון הזה הוא כמו פיטרסון, רק שהוא מסנכרן יותר מתהליך אחד
//בנוסף השינוי הוא הסרבול שמכניסים בשביל לשלב את המנעול והמפתח והשימוש בפעולה החדשה

//do for each thread
while(true){
 //התהליך הנוכחי ממתין לכניסה לקטע קריטי
 //waiting - משותף לכל התרדים
 waiting[i] = true;
 //key - מפתח
 //נועלים את המפתח. המשתנה הזה הוא נפרד לכל תהליכון
 //בהמשך נראה את lock
 //lock - מנעול
 //המנעול משותף לכל התרדים והוא מסנכרן ביניהם ומשפיע על הערך של המפתח
 key=1;
 //המתנה פעילה כל עוד המפתח נעול והתהליך בהמתנה
 while(waiting[i]&&key==1){
  //הפעולה CAS תבצע:
  //אם הערך של המנעול הוא 1 אז נשאיר אותו נעול ונחכה
  //אם הערך של המנעול הוא 0 אז נעדכן אותו להיות 1 - ננעל את המנעול המשותף
  //בכל מקרה ערך ההחזרה יהיה הערך המקורי של המנעול
  //ולכן אם המנעול היה פתוח אז נחזיר 0 ונישאר בהמתנה בתוך הלולאה
  //ואם המנעול היה נעול אז נחזיר 1 וניכנס לעוד איטרציה
  //ואם המנעול היה פתוח ועכשיו ננעלנו אותו אז נחזיר 0 ונצא מהלולאה
  //לסיכום: הלולאה מבצעת המתנה עד שהמנעול יהיה פתוח ואז נועלת אותו.
  //הפעם אפשר להיות בטוחים שהנעילה היא בגלל התהליכון שלנו
  key=CAS(&lock,0,1);
 }
 //עכשיו אנחנו כבר לא ממתינים - נכנסים לקטע הקריטי
 waiting[i]=false;
 /* critical section */
 //עכשיו אנחנו עוברים לחפש את התהליכונים הבאים ולבדוק אם הם ממתינים
 //מוצא את האינדקס של התהליכון הבא
 j=(j+1)%n;
 //עובר על כל התהליכונים ובודק אם הם ממתינים
 while((j!= i) && !waiting[j]){
  //מתקדם לתהליכון הבא
  j=(j+1)%n;
 }
 //אם חזרנו לתהליכון הנוכחי - סימן שכל התהליכים האחרים לא ממתינים ואפשר לשחרר את המנעול
 //ברגע שנשחרר את המנעול נצא החוצה ונסיים
 if(j==i){
  lock = 0;
 }
 //אם הגענו לתהליך אחר שכן ממתין אז נגדיר לו שיפסיק להמתין וינסה להיכנס לקטע הקריטי
 //כי עכשיו הוא כן פנוי - סיימנו תהליכון
 else{
  waiting[j]=false;
 }
/* remainder section */
}

//יש מניעה הדדית - על פי נכונות פיטרסון
//יש המתנה מוגבלת - עוברים על כל התהליכים האחרים ובודקים האם הם ממתינים
```